



同濟大學
TONGJI UNIVERSITY

算法设计与分析

王俊丽

junliwang@tongji.edu.cn

1、算法概述

- ▶ *Why study algorithms?*
- ▶ *Course overview*
 - ▶ *1.1 Algorithm and program*
 - ▶ *1.2 Algorithm complexity analysis*
 - ▶ *1.3 NP hard theory of algorithms*

1、算法概述

- ▶ *Why study algorithms?*
- ▶ *Course overview*
 - ▶ *1.1 Algorithm and program*
 - ▶ *1.2 Algorithm complexity analysis*
 - ▶ *1.3 NP hard theory of algorithms*

Why study algorithms?

Old roots, new opportunities.

- Study of algorithms dates at least to Euclid.

古希腊数学家欧几里得 数学史上不朽名著《几何原本》

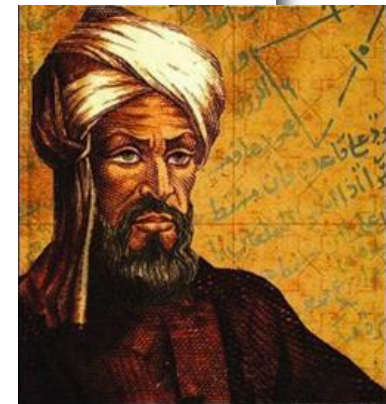
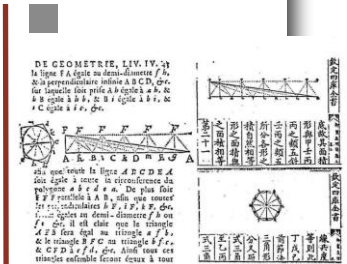
中国古代，算法称为“术” 《九章算术》(问, 答, 术)

- Etymology of “Algorithm”

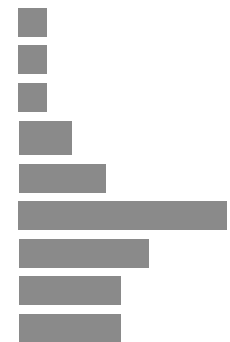
9世纪学者**花拉子密** (Al'Khwarizmi, 拉丁语名为 Algorismus, 公元780~850年) 发表了著名的《**印度算法**》一书，一种对世界产生难以估量影响的奇妙数字才为世人了解并接受。因此，人们把这种数字称作**阿拉伯数字**。他的思想在12世纪传入欧洲。

由他的名字Algorismus，派生出Algorithm

300 BCE



1920s
1930s
1940s
1950s
1960s
1970s
1980s
1990s
2000s



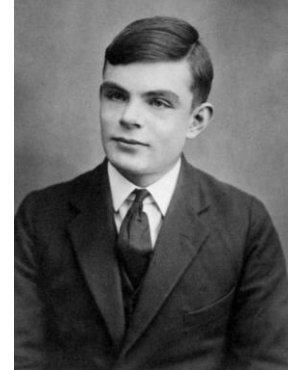
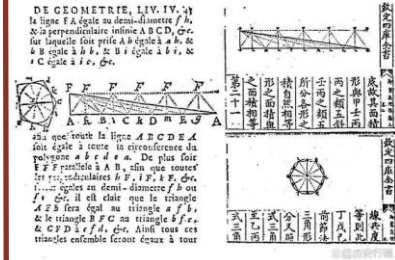
Why study algorithms?

Old roots, new opportunities.

- Study of algorithms dates at least to Euclid.
- Etymology of “Algorithm”
- Formalized by Church and Turing in 1930s.

可计算性理论是计算机理论的基础，而图灵机与丘奇-图灵论题是可计算性理论的基石，其意义之丰富，涵盖了数学、算法理论、数理逻辑以及哲学意义上的深刻内容

300 BCE



1920s
1930s
1940s
1950s
1960s
1970s
1980s
1990s
2000s

Why study algorithms?

They may unlock the secrets of life and of the universe.

Computational models are replacing math models in scientific inquiry.

```
for (double t = 0.0; true; t = t + dt)
  for (int i = 0; i < N; i++)
  {
    bodies[i].resetForce();
    for (int j = 0; j < N; j++)
      if (i != j)
        bodies[i].addForce(bodies[j]);
  }
```

21st century science
(algorithm based)

$$\begin{aligned} E &= mc^2 \\ F &= ma \end{aligned} \quad F = \frac{Gm_1m_2}{r^2}$$
$$\left[\left[-\frac{h^2}{2m} \nabla^2 + V(r) \right] \right] \Psi(r) = E \Psi(r)$$

20th century science
(formula based)

有两种思想，就像摆放在天鹅绒上的宝石那样熠熠生辉，一个是微积分，另一个就是算法。微积分以及在微积分基础上建立起来的数学分析体系造就了现代科学，而算法则造就了现代世界。

——大卫·柏林斯基 《算法的诞生》

Why study algorithms?

受过良好训练的计算机科学家知道怎样处理算法：如何构造算法、操作算法、理解算法以及分析算法。这些知识远不止为了写出良好的计算机程序而准备的。算法是一种一般性的智能工具，它必定有助于我们对其他学科的理解，不管是化学、语言学或音乐，还是其他学科。为什么算法会有这种作用呢？我们可以这样理解：人们常说，一个人只有把知识教给别人，才能真正掌握它。实际上，一个人只有把知识教给"计算机"，才能"真正"掌握它，也就是说，将知识表述为一种算法……比起简单按照常规去理解事物，尝试用算法将其形式化能使我们的理解更加深刻。

——唐纳德·克努特

1974年图灵奖获得者、计算机算法的奠基人，TEX发明人



1970s
1980s
1990s
2000s

- Some important algorithms were discovered

Why study algorithms?

To **solve problems** that could not otherwise be addressed.

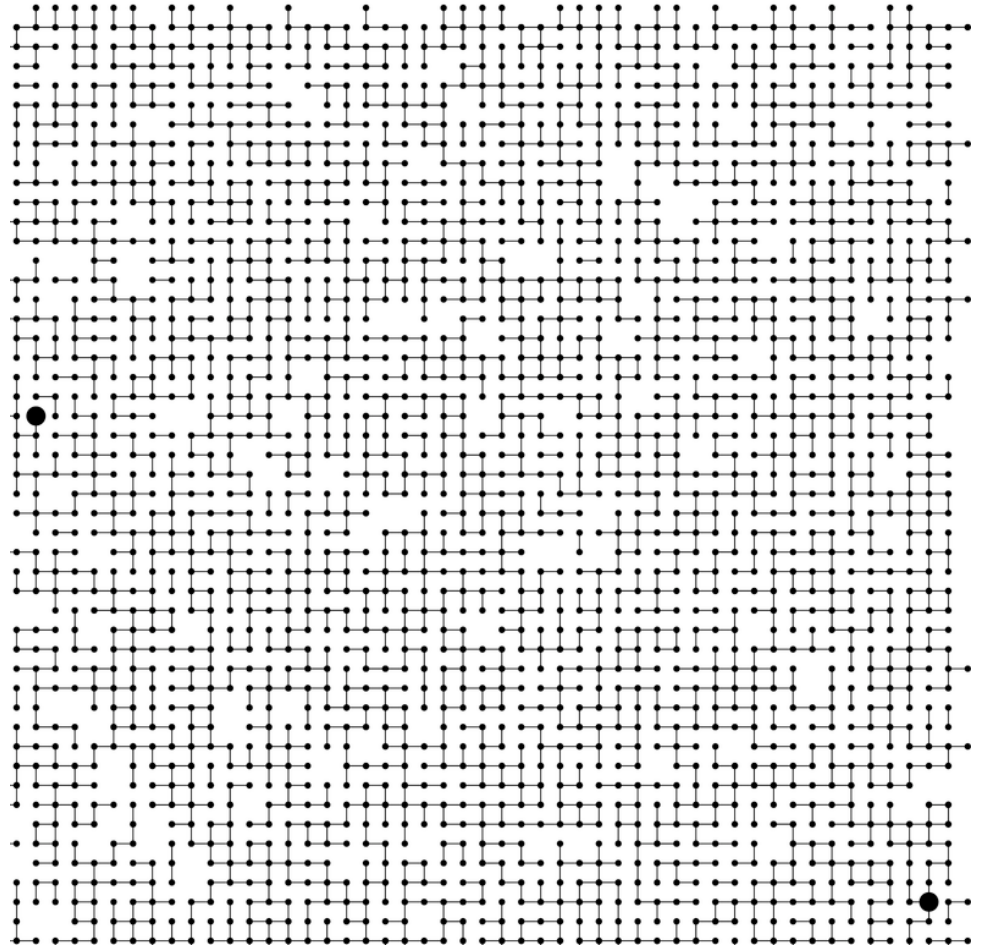
Ex. Network connectivity.

Social Network

Computer Network

Transportation Network

.....



Why study algorithms?

Their impact is broad and far-reaching.

Internet. Web search, packet routing, distributed file sharing, ...

Biology. Human genome project, protein folding, ...

Computers. Circuit layout, file system, compilers, ...

Computer graphics. Movies, video games, virtual reality, ...

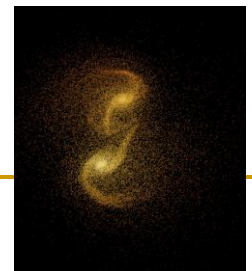
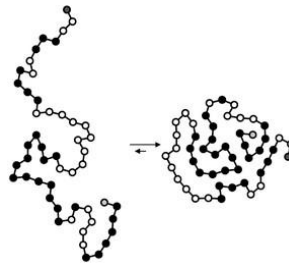
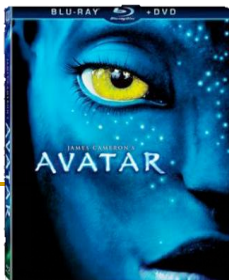
Security. Cell phones, e-commerce, voting machines, ...

Multimedia. MP3, JPG, DivX, HDTV, face recognition, ...

Social networks. Recommendations, news feeds, advertisements, ...

Physics. N-body simulation, particle collision simulation, ...

⋮



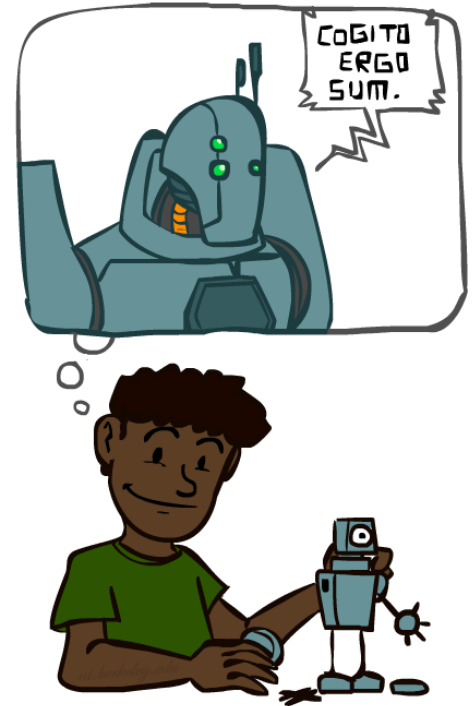
Why study algorithms?

Artificial Intelligence

算法

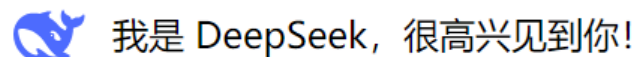
数据

算力



Why study algorithms?

For fun and profit.



我可以帮你写代码、读文件、写作各种创意内容, 请把你的任务交给我吧~



1、算法概述

- ▶ *Why study algorithms?*
- ▶ *Course overview*
 - ▶ *1.1 Algorithm and program*
 - ▶ *1.2 Algorithm complexity analysis*
 - ▶ *1.3 NP hard theory of algorithms*

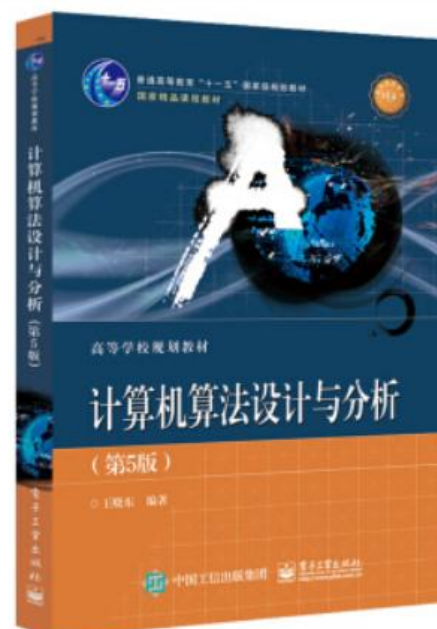
课程目标

- 了解计算机应用中的各种常用算法
- 掌握设计各种算法的基本原理、方法和技巧
- 能运用所学到的知识熟练地分析各种算法，并能指出解决同一问题的各种算法的好坏。

Question: 算法好坏的标准是什么？

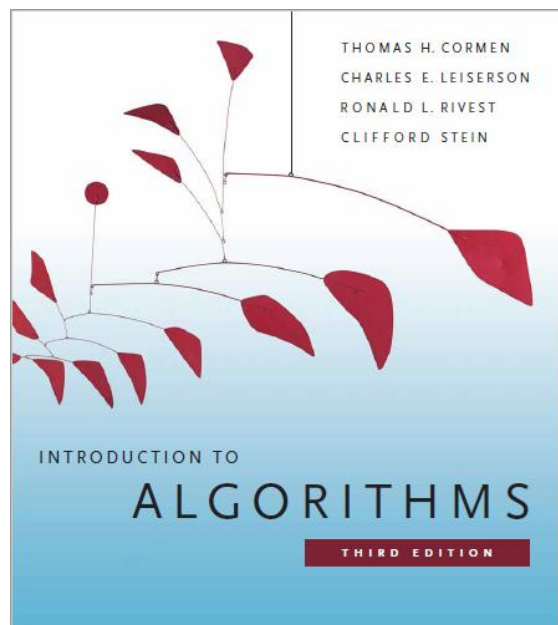
教材

- 王晓东，计算机算法设计与分析（第5版），电子工业出版社，2018



参考教材

- Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein, Introduction to Algorithms, Third Edition, The MIT Press



课程知识图谱

第1章 算法概述

算法设计、算法复杂性分析

算法设计策略

第2章 递归与分治策略

第3章 动态规划

第4章 贪心算法

第5章 回溯法

第6章 分支限界法

经典问题

排序与查找

矩阵运算

背包与装载

日程安排与调度

搜索问题

最短路径

算法分析

正确性证明

时间复杂度 O

空间复杂度

第8章 线性规划与网络流

线性规划问题 单纯形算法 最大网络流问题

课程考核

- 考勤 10%
- 平时作业 30%
- 期中考试（大作业） 10%
- 期末考试 50%

作业要求

- 命名格式：学号+姓名+题目名称.xxx。作业提交Canvas
- 算法类的题先给出问题分析、算法思想、算法伪代码、复杂度分析等；
- 实现题提交关键代码文件*.cpp, *.h (不要把IDE整个工程文件打包)以及输入、输出文件(或界面)

伪代码示例（选择排序）

输入: 数组 $A[a_1, a_2, \dots, a_n]$

输出: 升序数组 $A'[a'_1, a'_2, \dots, a'_n]$, 满足 $a'_1 \leq a'_2 \leq \dots \leq a'_n$

```
for  $i \leftarrow 1$  to  $n - 1$  do
     $cur\_min \leftarrow A[i]$ 
     $cur\_min\_pos \leftarrow i$ 
    for  $j \leftarrow i + 1$  to  $n$  do
        if  $A[j] < cur\_min\_pos$  then
            //记录当前最小值及其位置
             $cur\_min \leftarrow A[j]$ 
             $cur\_min\_pos \leftarrow j$ 
        end
    end
    交换  $A[i]$  和  $A[cur\_min\_pos]$ 
end
```

选择排序

- 第一次遍历找到最小元素
- 第二次在剩余数组中遍历找到次小元素
- ...
- 第 n 次在剩余数组中遍历找到第 n 小元素

1、算法概述

- ▶ *Why study algorithms?*
- ▶ *Course overview*
 - ▶ *1.1 Algorithm and program*
 - ▶ *1.2 Algorithm complexity analysis*
 - ▶ *1.3 NP hard theory of algorithms*

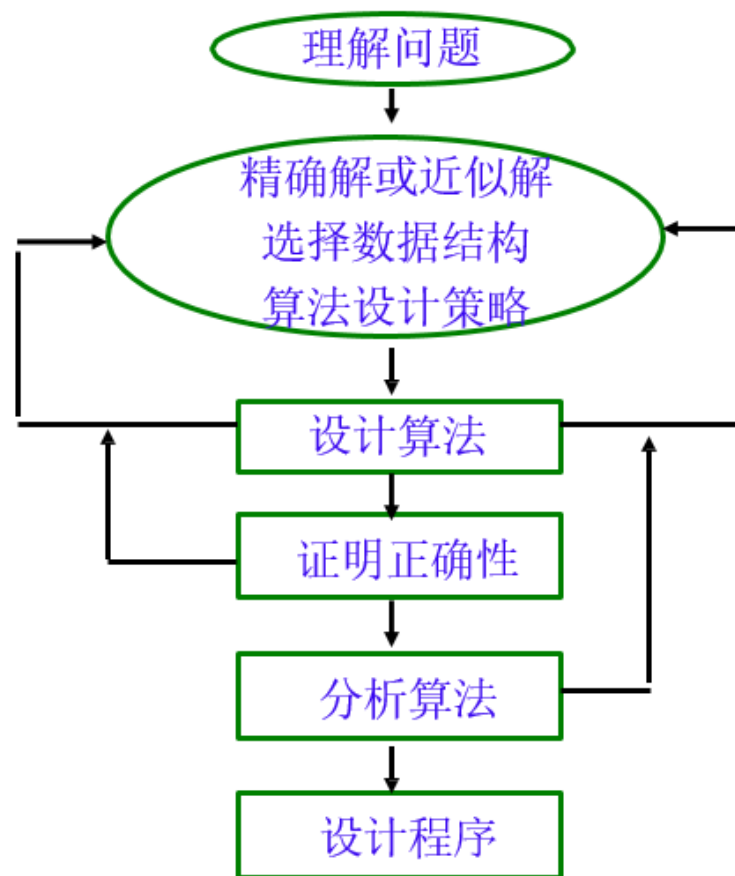
学习要点:

- 理解算法的概念。
- 理解什么是程序，程序与算法的区别和内在联系。

1.1 算法(Algorithm)

- **算法**是指解决问题的一种方法或一个过程。算法是若干指令的有穷序列，满足性质：

- (1)**输入**：有外部提供的量作为算法的输入
- (2)**输出**：算法产生至少一个量作为输出
- (3)**确定性**：组成算法的每条指令是清晰，无歧义的
- (4)**有限性**：算法中每条指令的执行次数是有限的，执行每条指令时间也是有限的。



1.1 算法(Algorithm)

例：最大公约数

两个不全为 0 的非负整数 m 和 n 的最大公约数是能够同时整除它们的最大正整数，**记为** $\gcd(m,n)$

$$\gcd(60,24) = 12$$

算法(Algorithm)

算法1： 中学时: 基于因式分解

计算 $\gcd(60, 24)$:

$$60 = 2 \times 2 \times 3 \times 5$$

$$24 = 2 \times 2 \times 2 \times 3$$

$$\gcd(60, 24) = 2 \times 2 \times 3 = 12$$

第一步： 计算 m 的所有质因数。

第二步： 计算 n 的所有质因数。

第三步： 从第一步和第二步求得的质因数分解式中找到所有的公因数。

第四步： 将公因数相乘， 其结果作为给定数字的最大公约数。

算法(Algorithm)

算法2：基于最大公约数的定义：m和n的最大公约数就是能够同时整除它们的最大正整数。

第一步：将 $\min\{m, n\}$ 的值赋给 t。

第二步：m除以t。如果余数为 0，进入第三步；否则进入第四步。

第三步：n除以t。如果余数为 0，**返回t值作为结果**；否则进入第四步。

第四步：把 t 的值减1。返回第二步。

当一个输入为0时？

1.1 算法(Algorithm)

算法3：基于递归的思想设计的gcd算法

欧几里得算法 (Euclid's algorithm)

$$\text{gcd}(m, 0) = m$$

$$\text{gcd}(m, n) = \text{gcd}(n, m \bmod n)$$

// $m \bmod n$ 表示 m 除以 n 之后的余数

$$\text{gcd}(60, 24) = \text{gcd}(24, 12) = \text{gcd}(12, 0) = 12$$

算法(Algorithm)

欧几里得算法 (Euclid's algorithm)

第一步：如果 $n=0$ ，返回 m 的值作为结果，过程结束；
否则，进入第二步。

第二步： m 除以 n ，将余数赋给 r 。

第三步：将 n 的值赋给 m ，将 r 的值赋给 n ，返回第一步

算法 Euclid(m,n) //使用欧几里得算法计算 $\gcd(m,n)$

//输入：两个不全为0的非负整数 m,n

//输出： m, n 的最大公约数

while $n \neq 0$ **do**

$r \leftarrow m \bmod n$

$m \leftarrow n$

$n \leftarrow r$

return m

Euclid(m, n)算法可以很容易地用于很大的数

1.1 程序(Program)

- 程序是算法用某种程序设计语言的具体实现。
- 程序可以不满足算法的性质(4)：有限性。
 - 例如操作系统，是一个在无限循环中执行的程序，因而不是一个算法。
 - 操作系统的各种任务可看成是单独的问题，每一个问题由操作系统中的一个子程序通过特定的算法来实现。该子程序得到输出结果后便终止。

Question

- 具体问题:

- 从100本书中找到某一本，用什么方法？

- 通用问题:

- 从 n 本书中找到某一本，用什么方法？

分治法找书算法

1. 排序书籍
2. 确定中间书
3. 比较目标书与中间书，确定目标书位于中间书的左边还是右边。
4. 将当前的搜索范围缩小后，重复上述步骤，直到找到目标书或确定其不存在。
5. 当搜索范围缩小到单本书时，如果该书为目标书，则返回该书；否则，返回目标书不存在。

如何证明其正确性

- **数学归纳法**适用于证明涉及自然数的命题
- 数学归纳法的三个步骤：
 - 基础步骤（基例）：证明命题对最小的输入成立。
 - 归纳假设（归纳假设）：假设命题对所有小于等于 n 的输入成立。
 - 归纳步骤（归纳证明）：证明命题对 $n+1$ 的输入也成立。

1、算法概述

- ▶ *Why study algorithms?*
- ▶ *Course overview*
 - ▶ *1.1 Algorithm and program*
 - ▶ *1.2 Algorithm complexity analysis*
 - ▶ *1.3 NP hard theory of algorithms*

Integer Multiplication

Problem(問): 44

x 97

Answer(答): 4268

Solution(術)?

Integer Multiplication

Problem(問):

$$\begin{array}{r} \text{16} \\ \overbrace{1234567895931413}^{16} \\ \times 4563823520395533 \\ \hline \end{array}$$

Integer Multiplication

n

1233925720752752384623764283568364918374523856298
X 4562323582342395285623467235019130750135350013753

???

How fast is the grade-school multiplication algorithm?

(How many one-digit operations?)

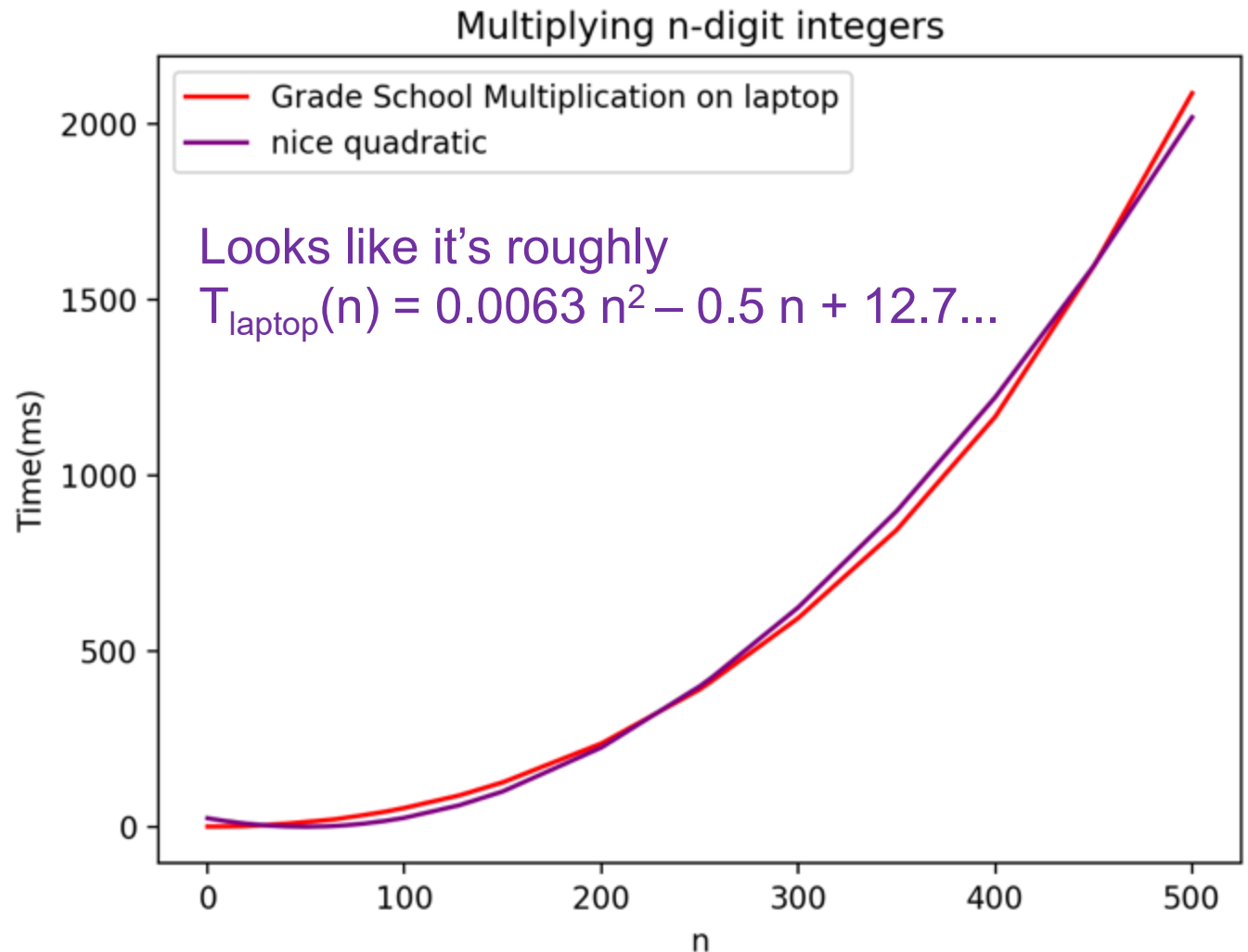
About n^2 one-digit operations

At most n^2 multiplications,
and then at most n^2 additions (for carries)
and then add n different $2n$ -digit numbers...

Grade-School Multiplication “runs in time $O(n^2)$ ”

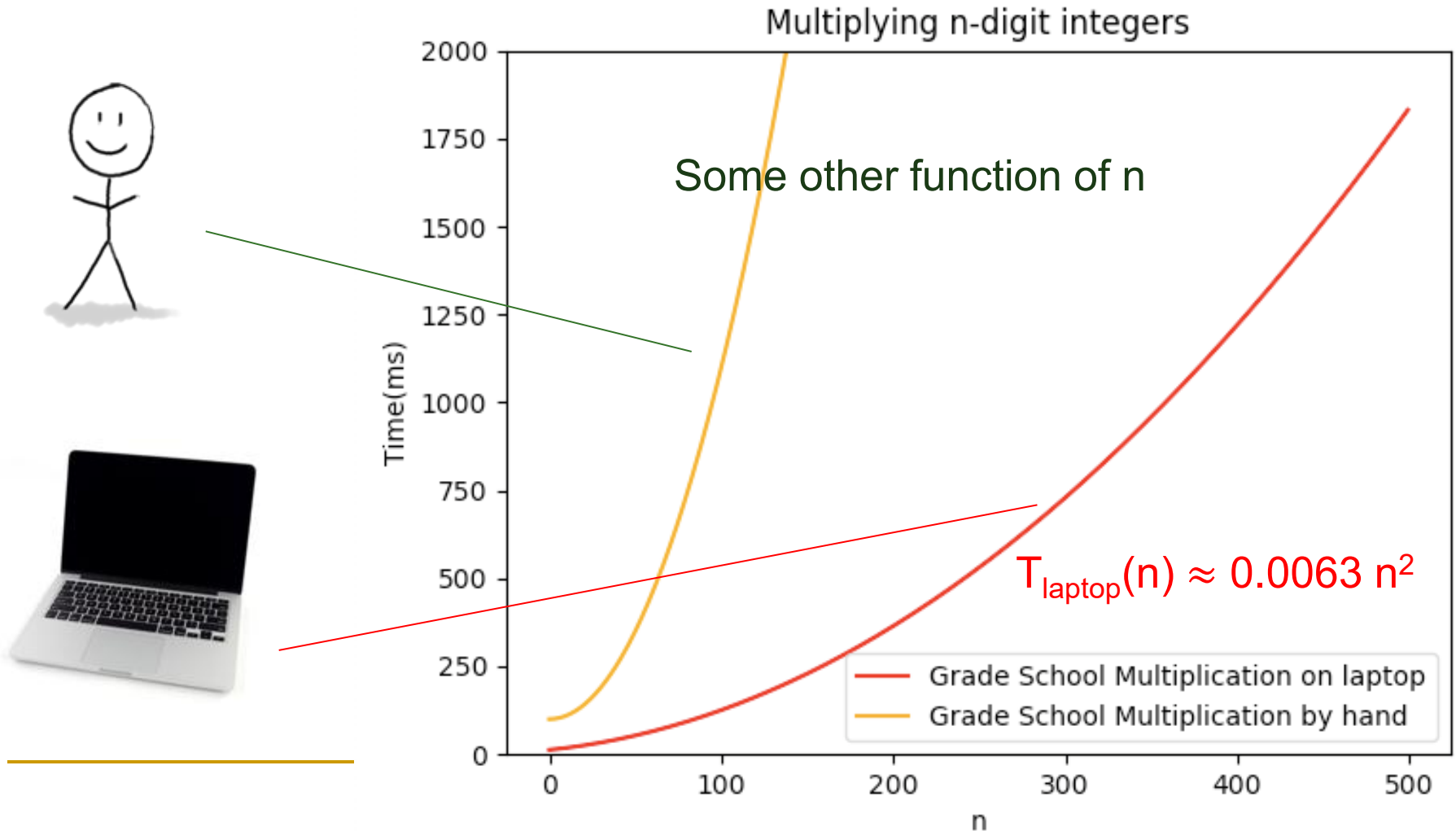
Implemented on a laptop

The runtime “scales like” n^2

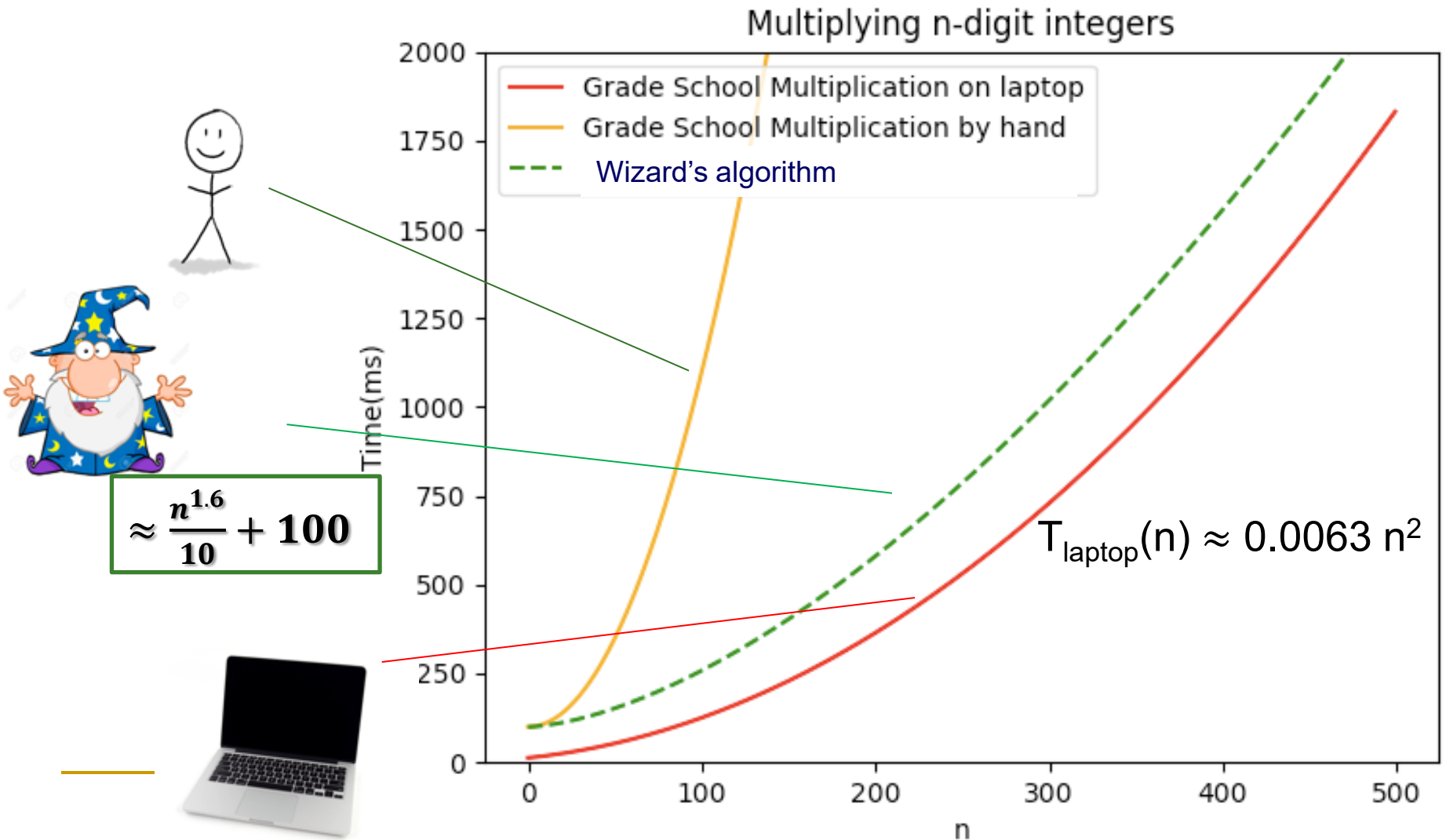


Implemented by hand

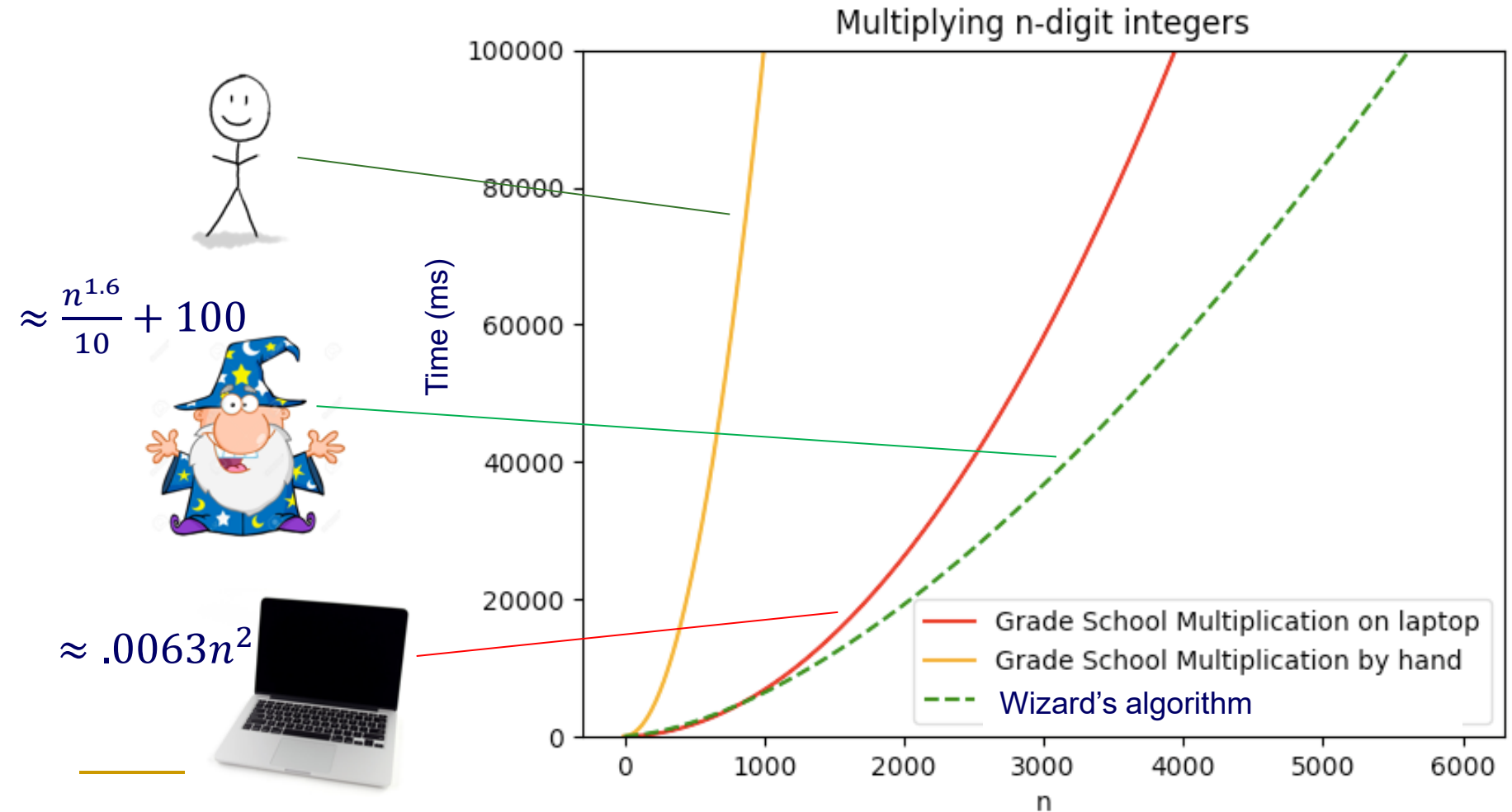
The runtime still “scales like” n^2



Why is big-Oh notation meaningful?



Let n get bigger...



Can we do better?

- An algorithm that runs in time $O(n^{1.6})$ is “better” than an algorithm that runs in time $O(n^2)$.
- So the question is **Can we do better?**

学习要点:

- 掌握算法的计算复杂性概念。
- 掌握算法渐近复杂性的数学表述。
- 掌握算法分析的基本法则。
- 掌握算法分析中常用的复杂函数。

算法复杂性分析

- 算法复杂性 = 算法所需要的计算机资源
- 算法的时间复杂性 $T(n)$;
- 算法的空间复杂性 $S(n)$ 。

其中 n 是问题的规模（输入大小）。

当给定问题有多种算法时，选择复杂度最低者是选用算法遵循的一个重要原则。

1.2 算法的时间复杂性

和算法输入 I 有关

(1) 最坏情况下的时间复杂性

$$T_{\max}(n) = \max\{ T(I) \mid \text{size}(I)=n \}$$

(2) 最好情况下的时间复杂性

$$T_{\min}(n) = \min\{ T(I) \mid \text{size}(I)=n \}$$

(3) 平均情况下的时间复杂性

加权平均时间复杂度

$$T_{\text{avg}}(n) = \sum_{\text{size}(I)=n} p(I)T(I)$$

其中 I 是问题的规模为 n 的实例， $p(I)$ 是实例 I 出现的概率。

顺序查找算法复杂性?

算法分析方法

例：顺序搜索算法

```
template<class Type>
int seqSearch(Type *a, int n, Type k)
{
    for(int i=0; i<n; i++)
        if (a[i] == k) return i;
    return -1;
}
```

时间复杂度？

$$T_{\max}(n)$$

$$T_{\min}(n)$$

$$T_{\text{avg}}(n)$$

顺序查找算法复杂性

(1) $T_{\max}(n) = \max\{ T(I) \mid \text{size}(I)=n \} = O(n)$

(2) $T_{\min}(n) = \min\{ T(I) \mid \text{size}(I)=n \} = O(1)$

(3) 在平均情况下，假设：

(a) 搜索成功的概率为 p ($0 \leq p \leq 1$)；

(b) 在数组的每个位置 i ($0 \leq i < n$) 搜索成功的概率相同，均为 p/n 。

$$\begin{aligned} T_{\text{avg}}(n) &= \sum_{\text{size}(I)=n} p(I)T(I) \\ &= \left(1 \cdot \frac{p}{n} + 2 \cdot \frac{p}{n} + 3 \cdot \frac{p}{n} + \cdots + n \cdot \frac{p}{n} \right) + n \cdot (1 - p) \\ &= \frac{p}{n} \sum_{i=1}^n i + n(1 - p) = \frac{p(n+1)}{2} + n(1 - p) = O(n) \end{aligned}$$

1.2 算法的时间复杂性

- 可操作性最好且最有实际价值的是**最坏情况**下的复杂性分析
- 如果算法的**最好情况**下的复杂性都无法满足要求，则不需要考虑
- **平均情况**复杂性虽然有意义，但是难以计算，需要分析算法在各种输入上的概率分布

当规模逐渐趋向于一个极大数时的算法复杂度？

本周小结

- 算法是解决问题的一种方法或一个过程，若干指令组成的有穷序列。
- 有两种算法效率：时间效率和空间效率。时间效率指出算法运行得有多快；空间效率涉及算法需要的额外空间。
- 算法的时间效率主要用它输入规模的函数来度量，该函数计算算法基本操作的执行次数。
- 对于有些算法来说，对于相同规模的输入，它的运行时间会有相当大的不同，导致了最差效率、平均效率和最优效率等概念的产生。
- 当算法的输入规模趋向于无穷大时，算法的运行时间表现出固定的增长次数。我们建立的分析算法时间效率的框架，主要就是基于这个增长次数。